Classical keypoint extraction on your 30 images

Step-by-step guide: label in CVAT, run the method, test it, report. Task: find 5 keypoints (head, left wing, right wing, tail, centre) of the aircraft with classical image processing: no CNN, no training, no colour detection. Your CVAT labels are only the **answer key** used to measure how good the method is.

Honest status: the code was tested on synthetic images, and by eye on 4 real Banshee photos. It has not been scored on real labels yet, and the CVAT reader has not seen a real CVAT export. Expect weak results on side and oblique views. Reporting that frankly is a good result.

The big picture

- **Phase 1 (development):** you label the dev photos; the code predicts the points; you compare, look at the pictures, and improve the method.
- **Phase 2 (final test):** you run once on test photos that you never used while improving. That number is your honest result.
- **Later (deployment):** your team's detector gives the box, the classical method gives the 5 points. No labels needed.

What the method does on one image

1. Take the aircraft box (from CVAT now, from the team's detector later).
2. **GrabCut** outlines the aircraft inside the box. It learns background vs aircraft from the image itself, so no fixed colour is assumed.
3. **Straighten** the outline along its main axis (PCA / image moments). The thinner end is guessed to be the head.
4. Read the points from the outline by rules: head and tail = the two ends, wings = the side extremes, centre = the centre of the shape.
5. Planned improvements (to be tested on your dev images): snap points to real corners (Shi-Tomasi / FAST) and carry points over from labelled photos with SIFT matching.

Part A: Prepare the photos

1. Make a folder **banshee/** and put the 30 photos in it. Rename them b01.jpg ... b30.jpg.
2. Split them: **dev/** with 20 photos and **test/** with 10 photos.
3. Put photos of the same aircraft or the same event in only one of the two folders. Otherwise the test looks better than it really is.
4. Do not look at the test photos' results while you develop. They are for one final run.

```
banshee/
  dev/   b01.jpg ... b20.jpg
  test/  b21.jpg ... b30.jpg
```

Part B: Label in CVAT

Use cvat.ai (free account) or your team's CVAT. Do this twice: one task for dev, one for test.

1. Create a new task, for example **banshee_dev**.
2. Add 6 labels: **aircraft** (type Rectangle) and **head, left wing, right wing, tail, centre** (each type Points). Spelling is flexible (case and spaces do not matter), the order of the five is not important to the reader.
3. Upload the photos of that folder.
4. For every photo: draw a loose rectangle around the aircraft (label aircraft), then click one point for each of the five names.
5. Rules for clicking, to stay consistent:

• **head**: the very tip of the nose.

• **tail**: the rear end of the body along the centreline.

• **left wing / right wing**: the wingtips, using the aircraft's **own** left and right (pilot's view), not the left and right of the picture.

• **centre**: the middle of the body, where the centreline meets the wingtip-to-wingtip line.

• Same physical spot every time, from every angle. If a part is hidden, mark it outside / skip it. Do not guess.

6. Label the first 3 photos, then stop and check them against the rules above. Fix mistakes now, not after 30 photos.
7. Export: Task menu, **Export task dataset**, format **CVAT for images 1.1**, untick Save images. From the zip take **annotations.xml** and save it as **dev.xml** (and **test.xml** for the other task).

Checkpoint: send your first 5 photos and the XML for a check before labelling all of them. If the XML contains <track> instead of <image>, it was exported as a video; re-export as 'CVAT for images 1.1'.

Part C: Set up the code

1. Unzip **aircraft_kp_project.zip** and open a terminal in the folder **aircraft_kp**.
2. Install the libraries (Ubuntu):

```
python3 -m venv venv && source venv/bin/activate
pip install numpy opencv-python
```

If it asks for dlib, you do not need it for this task; tell your helper.

Part D: Run on the dev photos

```
python evaluate_cvat.py --images banshee/dev --cvat dev.xml --out outputs/dev
```

It reads your box from the XML, outlines the aircraft, predicts the 5 points and compares them with your labels. It prints:

- error per keypoint in pixels and as a percentage of the box diagonal (5% = small miss, 10% = big miss),
- the share of points within 5% and within 10%,
- how often left/right wings or head/tail were swapped.

Then open **outputs/dev/overlay.jpg**: coloured dots are the method, white rings are your labels. Look at the pictures before you trust any number. The file **per_image.csv** lists the error for each photo, so you can find the worst ones.

No boxes at all? Add **--auto-box**; the code finds the box with GrabCut on the whole image. It worked on 3 of my 4 test photos and was fooled by clutter on the 4th, so drawing boxes in CVAT is safer.

Try one photo quickly (no labels needed):

```
python demo_box.py --image banshee/dev/b01.jpg --box 75 80 280 152 --out outputs/one.jpg
python demo_box.py --image banshee/dev/b01.jpg --out outputs/one.jpg    # box found automatically
```

Part E: Improve using the dev result

1. Find the worst keypoint in the table (for example wings in side views) and the worst photos in per_image.csv.
2. Decide the cause by looking at overlay.jpg: bad outline (GrabCut leaked into the background), wrong end (head and tail swapped), or wrong wing rule.
3. Try one change at a time: corner snapping (Shi-Tomasi / FAST), SIFT transfer from your labelled photos, or **--mode edges**.
4. Re-run Part D after every change. **Keep a change only if the dev score improves.**
5. Never tune on the test photos.

Part F: Final test (once)

```
python evaluate_cvat.py --images banshee/test --cvat test.xml --out outputs/test
```

Run this only when you have finished changing the method. This is the number you report. If your team's CNN can give its predicted points for the same images, add **--cnn-csv cnn_points.csv** and both methods are scored in the same way on the same images.

Part G: Write-up for your team

- One table: error per keypoint, within 5% / 10%, swap rates (dev and test).
- 4 to 6 pictures from overlay.jpg: some good, some bad.
- One paragraph on why it fails where it fails (for example: wings overlap in side views, so the rule picks the wrong extremes; a flat shape seen obliquely does not show which end is the nose).
- One paragraph on what would help: more labelled data, a 3D model with solvePnP, motion direction in video, or a learned model if training becomes allowed.

If your team's labelled video frames arrive

- They are probably the same CVAT XML. Check whether it has <image> (fine) or <track> (video export, needs a small reader change).
- Frames are grey: GrabCut has only brightness to work with, so outlines may be weaker. Measure it.
- Split by video or flight pass, never by random frame (neighbouring frames are nearly identical).
- Ask whether detector boxes exist for those frames and use **--boxes boxes.csv** (columns: image,x1,y1,x2,y2).

The five algorithm names you heard in the team meeting

Name	What it is	Role here
GrabCut	Outlines an object inside a box	Step 2, finds the aircraft shape
PCA / moments	Finds the main axis of a shape	Step 3, straightens the aircraft
Shi-Tomasi / FAST / Harris	Find corners (unnamed)	Planned: snap wingtips, nose, tail to corners
SIFT	Describes a patch so it can be matched	Planned: copy points from a labelled photo
Viola-Jones	A trained detector (needs training)	Not used; your team's detector gives the box

None of these names the keypoints by itself. The rules on the outline decide which point is head, wing or tail.